

Student

I've built dimensions and facts. But real customers **move**, products get **recategorized**, employees change **departments**. What happens to my model?

ChatGPT • AI Assistant

Welcome to **Slowly Changing Dimensions (SCD)** - the single most important operational concept in dimensional modeling. This chapter teaches:

1. What an SCD actually is and why the name has "slowly."
2. **Type 0** - retain original (never change).
3. **Type 1** - overwrite (no history).
4. **Type 2** - add a new row (full history - the workhorse).
5. **Type 3** - add a column for previous value.
6. **Type 6** - hybrid (1 + 2 + 3 combined).
7. How to **choose the right SCD type** per attribute.
8. How to **implement SCD Type 2 in ETL** with a MERGE pattern.
9. A **hands-on customer-address change** walkthrough showing all four types side by side.

CONCEPT CARD

A **Slowly Changing Dimension (SCD)** is a dimension whose attributes change over time - a customer moves, a product gets recategorized, a store changes region. SCD techniques (named by Ralph Kimball) tell you *how* to store those changes: whether to keep history, overwrite, snapshot, or hybridize.

The name says "slowly" because dimensions change **rarely** compared to facts (which get millions of new rows daily).

SUMMARY CARD

By the end of this chapter you'll be able to:

- Pick the right SCD type for any dimension attribute.
- Implement SCD Type 2 with effective dates and current flags.
- Write a MERGE-based ETL that closes old rows and inserts new ones.
- Explain to any interviewer why the surrogate key exists in the first place.

[AI Prompt] Warm-up (paste into any AI chatbot): *"Explain slowly changing dimensions (SCD) with a story of a customer named Priya who moves from Hyderabad to Bengaluru. Show what happens under SCD Type 1 vs SCD Type 2, and why Type 2 preserves historical revenue reports correctly."*

Type 1 — Overwrite

Row	City
1	city="Hyderabad"

Change →

Row	City
1	city="Bengaluru"

Type 2 — Add Row

Surrogate Key	City	is_current=Key
CustomerD1	city="Hyderabad"	is_current=false

Surrogate Key	City	is_current=Key
CustomerD2	city="Bengaluru"	is_current=true

Type 3 — Add Column

Customer	current_city	previous_city
1	current_city="Bengaluru"	previous_city="Hyderabad"

Type 6 — Hybrid

Surrogate Key	City	current_city
CustomerR1	city="Hyderabad"	"Bengaluru"

Surrogate Key	City	current_city
CustomerR2	city="Bengaluru"	"Bengaluru"

Same event, four different storage decisions — each with different trade-offs.

SCD Types at a Glance — How the Same Change Looks Under Each Type.

SCD Types at a Glance — How the Same Change Looks Under Each Type

SCD Types Reference			
Type	Behavior	History Kept?	Use When
Type 0	Retain original — never change	Original only	Immutable attributes (birthdate, signup date)
Type 1	Overwrite the value	None	Corrections, or history doesn't matter
Type 2	Add a new row (versioning)	Full history	Business needs point-in-time accuracy
Type 3	Add a column for previous value	One prior value	Compare current vs one prior state
Type 4	Move history to a separate table	In separate history dim	Very frequently changing attributes
Type 6	Hybrid — Type 1 + Type 2 + Type 3	Full + current view	Need both 'as-was' and 'as-is'

sql – SCD Type 2 dimension – the workhorse structure

```
CREATE TABLE dim_customer (
  -- Surrogate PK: unique per VERSION of a customer, not per person
  customer_key      BIGINT      PRIMARY KEY,

  -- Natural key: the same across all versions of one person
  customer_id       VARCHAR(50) NOT NULL,

  -- Descriptive attributes (may change over time)
  full_name         VARCHAR(200),
  email             VARCHAR(200),
  city              VARCHAR(100),
  country           VARCHAR(80),
  segment           VARCHAR(50),

  -- SCD Type 2 columns – the versioning machinery
  effective_from    DATE        NOT NULL,
  effective_to      DATE,        -- NULL = still
  active
  is_current        BOOLEAN     NOT NULL DEFAULT TRUE,
  version_number    INT         NOT NULL DEFAULT 1,

  -- Audit
  updated_ts        TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Only one row per natural key can be "current" at a time
CREATE UNIQUE INDEX ux_dim_customer_current
  ON dim_customer(customer_id)
  WHERE is_current = TRUE;

-- Speed up point-in-time lookups
CREATE INDEX idx_dim_customer_natural_dates
  ON dim_customer(customer_id, effective_from, effective_to);
```

WHAT IS A SLOWLY CHANGING DIMENSION?

Give me a precise definition of an **SCD**.

ChatGPT • AI Assistant

A **Slowly Changing Dimension** is any dimension table whose **attribute values change over time**, and where the modeling team has to **decide how to store those changes**. It's called *slowly* because dimensions change on the order of hours or days, not milliseconds - unlike facts, which get millions of inserts.

CONCEPT CARD

What triggers an SCD decision?

1. A customer **moves** to a new city.
2. A product gets **recategorized** by marketing.
3. An employee **transfers** to another department.
4. A store's **manager** changes.
5. A supplier gets **acquired** and renamed.
6. A price tier gets **restructured**.

Every one of these forces the question: *"When the fact table joins to this dimension, which version do we want to see?"*

Student

Why does the version matter?

ChatGPT • AI Assistant

EXAMPLE CARD

Priya buys a laptop in **March**, when she lives in **Hyderabad**. She moves to **Bengaluru** in **July**. In November someone runs the report *"How much revenue came from Hyderabad customers in Q1?"*

Depending on your SCD choice:

1. **Type 1 (overwrite)** - Priya's city says Bengaluru; her March order shows up under Bengaluru. Q1 Hyderabad revenue is silently understated.
2. **Type 2 (versioning)** - Priya's March order joined to the Hyderabad version; Q1 Hyderabad revenue is historically accurate.

That single choice determines whether your KPIs are trustworthy.

🔗 ANALOGY

SCDs are like **photo albums for your data**. Type 1 says *"replace the old photo with the new one - I only want to see how she looks today."* Type 2 says *"keep every photo dated - I want to see how she looked in March vs how she looks now."*

⚠️ WARNING

Getting SCD wrong doesn't crash your system - it silently produces **wrong reports**. Executives lose trust when the same KPI gives different numbers each quarter.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Using Type 1 everywhere "because it's simple." A year later, someone asks "how has the customer segment mix evolved?" - and you can't answer.

Correct Way: Decide per-attribute. Some attributes need history (city, segment); some don't (email address you just corrected).

EXAM TIP CARD

Interview classic: *"Why is it called a slowly changing dimension?"* Winning answer: *"Because dimension attributes change on the order of hours to years, unlike facts which get millions of inserts per day. The 'slowly' contrasts the dimension update rate with the fact insert rate."*

🔥 EXERCISE

Look at any dimension table in your work (or the `dim_customer` from Chapter 2). For each column, ask: *"If this value changed tomorrow, would historical reports need the OLD or the NEW value?"* That's the SCD-type decision for that column.

[AI Prompt] Paste this into any AI chatbot: *"For a dim_customer with columns name, email, city, segment, and birth_date, tell me which SCD type is most appropriate for each column and why. Include the historical-report reasoning."*

❓ PRACTICE QUIZ

Why is 'slowly' in the name **Slowly Changing Dimension**?

Answer: Because dimensions change rarely (hours/days/years) compared to facts, which grow by millions of rows per day.

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

SCD TYPE 0 — RETAIN ORIGINAL

ChatGPT • AI Assistant

sql – Type 0 in practice – enforce immutability in the ETL

```
-- =====
-- Type 0 policy: birth_date and original_signup_date are
-- immutable. The MERGE below EXCLUDES them from UPDATE.
-- =====

CREATE TABLE dim_customer (
  customer_key      BIGINT      PRIMARY KEY,
  customer_id       VARCHAR(50) UNIQUE NOT NULL,
  full_name         VARCHAR(200),
  email             VARCHAR(200),

  -- Type 0 columns: NEVER updated after insert
  birth_date        DATE        NOT NULL,
  original_signup_date DATE      NOT NULL
);

-- MERGE that respects Type 0: birth_date and
-- original_signup_date are NOT in the UPDATE SET clause.
MERGE INTO dim_customer AS tgt
USING staging_customer AS src
  ON tgt.customer_id = src.customer_id
WHEN MATCHED THEN
  UPDATE SET
    full_name = src.full_name,
    email     = src.email
    -- birth_date and original_signup_date deliberately absent
WHEN NOT MATCHED THEN
  INSERT (customer_key, customer_id, full_name, email,
          birth_date, original_signup_date)
  VALUES (nextval('dim_customer_seq'), src.customer_id,
```

```
src.full_name, src.email,
src.birth_date, src.original_signup_date);
```

Student

What is **SCD Type 0**?

ChatGPT • AI Assistant

Type 0 is the simplest - the value **never changes** after it's first inserted. The original is *retained forever*. Any incoming update for that column is ignored.

CONCEPT CARD

When to use Type 0:

1. **Truly immutable attributes** - date of birth, original signup date, first-order date, VIN of a car.
2. **Legal / audit-required fields** - original agreement date, first name at signup (for verification).
3. **Historical anchors** - anything the business explicitly wants preserved from day one.

Student

Isn't Type 0 the same as Type 1 for values that never change?

ChatGPT • AI Assistant

EXAMPLE CARD

Almost - but the distinction is **intent**:

1. **Type 0** is a *policy decision* - "we will never allow this to change."
2. **Type 1** is a *mechanism* - "if it changes, overwrite."
3. A birthdate is Type 0: the ETL is designed to refuse updates.
4. A corrected typo in a name is Type 1: the ETL applies the update.

🗨️ ANALOGY

Type 0 is like the **birth date on a passport** - no matter how many times you renew it, that date is fixed forever. Type 1 is like the **photo** - it changes each renewal.

 WARNING

Type 0 doesn't mean the *entire row* is immutable - only specific columns. Other columns in the same dim can be Type 1, 2, 3, or 6.

 COMMON PITFALL / MISTAKE

Incorrect Way: Allowing a birth date to be silently overwritten in ETL. Now you've lost the truth about when someone was born.

Correct Way: Mark birthdate as Type 0 in your data catalog; ETL enforces "do not update this column."

EXAM TIP CARD

Interview classic: *"When is SCD Type 0 the right choice?"* Winning answer: *"For truly immutable attributes - birth date, VIN, original signup date. It's a policy decision that the value should never be updated, and the ETL is designed to reject updates."*

 EXERCISE

List 3 columns in your dimensions that should be Type 0. Then check your ETL - is anything silently overwriting them? Fix that.

[AI Prompt] Paste this into any AI chatbot: *"For dim_employee, dim_customer, dim_product, and dim_store, list 2 columns each that should be SCD Type 0. Justify each choice."*

SCD TYPE 1 — OVERWRITE

ChatGPT • AI Assistant

sql – SCD Type 1 – simple overwrite

```
-- =====
-- Type 1: email address was a typo – just correct it.
-- No history preserved. Simple UPDATE.
-- =====

-- Before
-- customer_key | customer_id | full_name | email
-- -----
-- 401          | C-123       | Priya K   | preya@x.com
```

```

UPDATE dim_customer
SET    email      = 'priya@x.com',
       updated_ts = CURRENT_TIMESTAMP
WHERE  customer_id = 'C-123';

-- After
-- customer_key | customer_id | full_name | email
-- -----
--    401       | C-123      | Priya K   | priya@x.com

-- Historical facts joined via customer_key still work,
-- but any downstream report NOW sees the corrected email.
-- That's fine – history has no value for a typo.

-- =====
-- Type 1 as part of a MERGE (typical ETL pattern)
-- =====

MERGE INTO dim_customer AS tgt
USING staging_customer AS src
      ON tgt.customer_id = src.customer_id
WHEN MATCHED THEN
      UPDATE SET
          email      = src.email,
          full_name   = src.full_name,
          updated_ts  = CURRENT_TIMESTAMP
WHEN NOT MATCHED THEN
      INSERT (customer_key, customer_id, full_name, email)
      VALUES (nextval('dim_customer_seq'),
              src.customer_id, src.full_name, src.email);

```

Student

What is **SCD Type 1**?

ChatGPT • AI Assistant

Type 1 is the simplest mechanism for changing values: **overwrite the old value with the new one**. No history is preserved. A simple **UPDATE** statement is all it takes.

CONCEPT CARD

When Type 1 is the right choice:

1. **Corrections** - the email address was misspelled; fix it.
2. **Attributes where history has no analytical value** - like a customer's preferred language.
3. **Fast-changing timestamp-y fields** - last *logindate* (updates constantly; history handled elsewhere).
4. **When storage matters more than history** - very large dimensions where every row is expensive.

Student

What's the downside?

ChatGPT • AI Assistant

⚠ WARNING

With Type 1, **you lose the ability to answer historical questions**. If a customer's segment was "new" in March and "vip" now, Type 1 makes their March order look like a VIP order retroactively. All aggregations by segment become "as-is" only.

EXAMPLE CARD

Type 1 in action:

1. Priya's email is `preya@x.com` . Typo. Correct to `priya@x.com` .
2. Simple `UPDATE dim_customer SET email='priya@x.com' WHERE customer_id='C-123'` .
3. No new row. No effective *from/effective* to update.
4. Every fact that ever joined to Priya's row now sees the correct email.

This is fine - historical reports don't need to see the typo.

💡 ANALOGY

Type 1 is like a **whiteboard**. Erase and write the new value. Simple. Fast. But once erased, the old value is gone.

⚠ COMMON PITFALL / MISTAKE

Incorrect Way: Using Type 1 on `customer_segment` because "it's easier." Six months later, marketing asks *"how has the segment mix evolved?"* and you can't answer.

Correct Way: Decide per attribute. Corrections → Type 1. Business-critical evolution → Type 2.

EXAM TIP CARD

Interview classic: *"When is Type 1 acceptable?"* Winning answer: *"For attributes where history has no analytical value - corrections, cosmetic changes, or fast-changing fields where snapshotting is handled elsewhere."*

EXERCISE

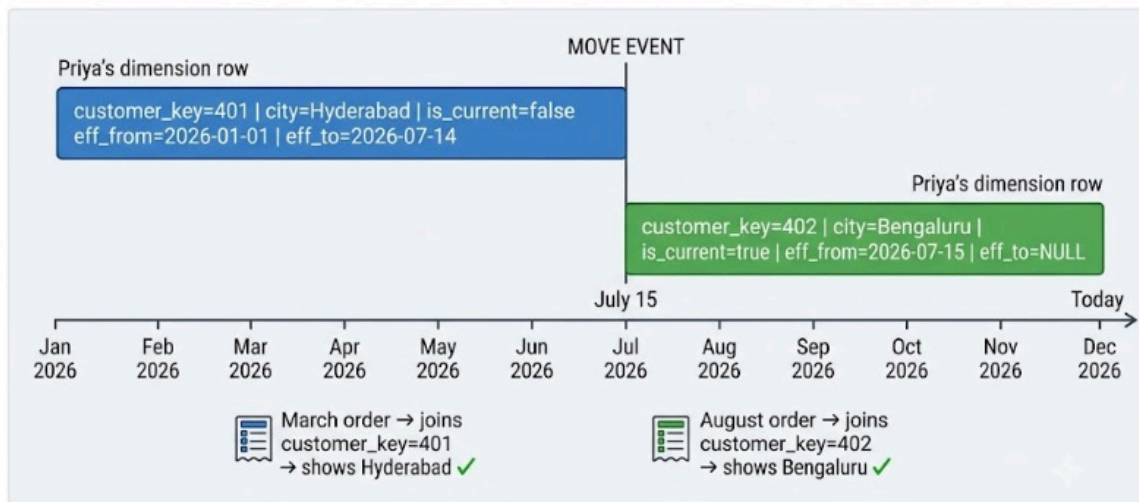
Look at your `dim_customer`. Pick 3 columns that would be fine as Type 1 (not "tracked historically"). For each, write the one-line business justification.

[AI Prompt] Paste this into any AI chatbot: *"Give me 5 dimension attributes where Type 1 (overwrite) is clearly the right SCD choice, and 5 where Type 1 would silently produce wrong historical reports. Justify each."*

SCD TYPE 2 — ADD ROW WITH HISTORY

ChatGPT • AI Assistant

SCD Type 2 in Action — Two Rows, Same Person, Different Versions



One customer, two dimension rows. Historical accuracy without ambiguity.

SCD Type 2 in Action — Two Rows, Same Person, Different Versions

sql — SCD Type 2 — close old row, insert new row

```
-- =====
-- Priya moves from Hyderabad to Bengaluru on 2026-07-15.
-- Type 2: close the old row and insert a new one.
-- =====
```

```

-- Step 1: Close the existing current row
UPDATE dim_customer
SET     effective_to = DATE '2026-07-14',
        is_current   = FALSE,
        updated_ts   = CURRENT_TIMESTAMP
WHERE  customer_id = 'C-123'
      AND is_current = TRUE;

-- Step 2: Insert the new current version with a NEW surrogate key
INSERT INTO dim_customer (
    customer_key, customer_id, full_name, email,
    city, country, segment,
    effective_from, effective_to, is_current, version_number
)
VALUES (
    nextval('dim_customer_seq'),
    'C-123', 'Priya K', 'priya@x.com',
    'Bengaluru', 'India', 'loyal',
    DATE '2026-07-15', NULL, TRUE,
    2                                -- version 2
);

-- =====
-- Result: two rows for the same customer_id, one closed,
-- one current. Historical facts join the correct version.
-- =====

-- 'As-is' report: always join on is_current = TRUE
SELECT c.full_name, c.city, SUM(s.net_amount) AS revenue
FROM   fact_sales s
JOIN   dim_customer c
      ON c.customer_key = s.customer_key
WHERE  c.is_current = TRUE
GROUP BY c.full_name, c.city;

-- 'As-was' report: point-in-time lookup
SELECT c.city, SUM(s.net_amount) AS revenue_q1
FROM   fact_sales s
JOIN   dim_customer c
      ON c.customer_id = 'C-123'
      AND s.order_date ≥ c.effective_from
      AND (s.order_date ≤ c.effective_to OR c.effective_to IS NULL)
WHERE  s.order_date BETWEEN DATE '2026-01-01' AND DATE '2026-03-31'
GROUP BY c.city;

```

Student

Show me **SCD Type 2** - the one everyone talks about.

ChatGPT • AI Assistant

Type 2 is the **workhorse** of dimensional modeling. When an attribute changes, you **do not update the existing row** - you **insert a new row with a new surrogate key**.

The old row is "closed" by setting `is_current = false` and stamping `effective_to`. Every historical fact still joins to its correct version.

CONCEPT CARD

Type 2's four columns you'll see on every dimension:

1. ``effective_from`` - the date this version became active.
2. ``effective_to`` - the date this version stopped being active (NULL = still active).
3. ``is_current`` - a boolean flag for fast "latest version" queries.
4. ``version_number`` - optional; a running counter of versions per natural key.

Student

Why do we insert a new row instead of updating?

ChatGPT • AI Assistant

EXAMPLE CARD

Because **each fact row already carries the surrogate key of the version it was joined with at insert time**:

1. Priya's row when she lived in Hyderabad had `customer_key = 401`.
2. Priya moves to Bengaluru. New row inserted: `customer_key = 402`, `city='Bengaluru'`, `is_current=true`.
3. Priya's March order is still linked to `customer_key = 401` → city = Hyderabad ✓
4. Priya's August order is linked to `customer_key = 402` → city = Bengaluru ✓
5. "Revenue by Hyderabad customers in Q1" is historically accurate.

This is the entire reason we use surrogate keys. Without them, Type 2 is impossible.

🔗 ANALOGY

Type 2 is like **version control for your dimension**. Each change is a new commit - you can check out any point in time. Type 1 is like editing a shared Google Doc - no history, whoever saw the old version has no way back.

⚠️ WARNING

Type 2 makes dimensions grow. A customer who moves 5 times has 5 rows. Plan storage accordingly (still cheap in modern warehouses - dimensions rarely exceed millions of rows even with heavy Type 2).

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Updating an existing row in place. Historical facts now retroactively report the new city - all past reports become wrong.

Correct Way: Close the old row (`is_current=false` , set `effective_to`), insert a new row with a new surrogate key.

EXAM TIP CARD

Interview classic: *"How does SCD Type 2 preserve history?"* Winning answer: *"Each change creates a new dimension row with a new surrogate key. Historical facts remain joined to their original surrogate key - so 'as-was' reports stay accurate. The is_current flag makes 'as-is' reports easy too."*

🔥 EXERCISE

Take your `dimcustomer`. Add the four SCD Type 2 columns. Simulate Priya moving: insert two rows for the same `customerid` - one closed, one current. Run a query that joins to only the current version.

[AI Prompt] Paste this into any AI chatbot: *"Show me the full SCD Type 2 update flow for a customer who changes address. Include the UPDATE (close old row), the INSERT (new row), and a SELECT that returns the version active on a specific date."*

SCD TYPE 3 — ADD COLUMN FOR PREVIOUS VALUE

ChatGPT • AI Assistant

```
sql - SCD Type 3 - one prior value stored alongside the current
```

```
-- =====
-- Type 3: dim_sales_territory. Region names change on
-- 2026-07-01. We want reports both ways for 6 months.
-- =====

CREATE TABLE dim_sales_territory (
    territory_key          BIGINT          PRIMARY KEY,
    territory_id           VARCHAR(20)     UNIQUE NOT NULL,
    territory_name         VARCHAR(100)    NOT NULL,

    -- Type 3 columns for the region attribute
    region                 VARCHAR(50),    -- current
    previous_region        VARCHAR(50),    -- one prior
    region_changed_on      DATE            -- when the
change happened
);

-- Before the change
-- territory_key | territory_id | region      | previous_region |
changed_on
-- ----- | ----- | ----- | ----- |
-- 77      | T-EAST-01  | East       | NULL           | NULL

-- Type 3 update: copy current → previous, then set current
UPDATE dim_sales_territory
SET    previous_region = region,          -- save the old one
       region         = 'East Central',  -- set the new one
       region_changed_on = DATE '2026-07-01'
WHERE territory_id     = 'T-EAST-01';

-- After the change
-- territory_key | territory_id | region      | previous_region |
changed_on
-- ----- | ----- | ----- | ----- | ---
-- 77      | T-EAST-01  | East Central | East           |
2026-07-01

-- Reports can now group by either OLD or NEW region
SELECT COALESCE(t.previous_region, t.region) AS old_region,
       t.region                             AS new_region,
       SUM(s.net_amount)                    AS revenue
FROM   fact_sales s
JOIN   dim_sales_territory t
```

```
ON    t.territory_key = s.territory_key
WHERE s.order_date ≥ DATE '2026-01-01'
GROUP BY old_region, new_region
ORDER BY revenue DESC;
```

Student

What is **SCD Type 3**?

ChatGPT • AI Assistant

Type 3 adds a **new column** to store the previous value, alongside the current one. You keep the current value in the original column, and store the immediately prior value in a **previous_** column.

It's a limited history - usually one or two prior values, not the full timeline.

CONCEPT CARD

Type 3's structure:

1. Existing column: **current_region** (or just **region**).
2. Add a new column: **previous_region**.
3. Optionally: a **previous_region_change_date** to record when it changed.
4. On change: copy current → previous, then update current.

Student

When is Type 3 useful?

ChatGPT • AI Assistant

EXAMPLE CARD

Type 3 shines in three narrow scenarios:

1. **Territory re-alignments** - sales region boundaries change; you need reports both ways (old territories AND new territories) for a transition period.
2. **Product reclassifications** - a product moves from category A to B; you want to run "as-was category" reports without full Type 2 complexity.

3. **Attribute renames** - organizational restructure; report by old and new names simultaneously.

Type 3 is essentially *"remember one prior value alongside the current one"* - cheaper than Type 2 for cases where you only need before/after, not full history.

🔗 ANALOGY

Type 3 is like a **before/after photo pair** on the same page. You can see both simultaneously, but you can't see 5 photos back - that would need Type 2.

⚠️ WARNING

Type 3 is rarely enough. If the business asks *"how did segments evolve over 3 years?"*, Type 3 (with only one prior value) can't answer. Reach for Type 2 when full history matters.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Using Type 3 for a heavily-changed attribute like customer city. After a few moves, you've lost most of the history and can't answer point-in-time questions.

Correct Way: Type 3 for rare, well-defined transitions ("we just re-aligned sales territories once"). Type 2 for attributes that change repeatedly.

EXAM TIP CARD

Interview classic: *"When would you pick Type 3 over Type 2?"* Winning answer: *"When the business needs a limited 'before/after' comparison for a one-off change like a sales-territory re-alignment or product reclassification - and Type 2's full-history overhead isn't justified."*

🔥 EXERCISE

Design a Type 3 solution for a product-category rename. Add `previous_category` and `previous_category_change_date` columns. Simulate a change and write a query that returns both old and new category revenue side by side.

[AI Prompt] Paste this into any AI chatbot: *"Show me DDL and an UPDATE for SCD Type 3 on dim_sales_territory where the region column changes. Include a before/after example and a query that reports revenue under both old and new region names."*

SCD TYPE 6 — HYBRID (1 + 2 + 3)

sql – SCD Type 6 – Type 2 versioning + Type 1 current + Type 3 previous

```
-- =====
-- Type 6 on dim_customer.city
-- =====

CREATE TABLE dim_customer (
    customer_key      BIGINT      PRIMARY KEY,
    customer_id       VARCHAR(50) NOT NULL,
    full_name         VARCHAR(200),

    -- Type 2 column: the point-in-time value
    city              VARCHAR(100),

    -- Type 1 column: the LATEST value, overwritten on every row
    current_city       VARCHAR(100),

    -- Type 3 column: the immediately prior city (per row)
    previous_city      VARCHAR(100),

    -- Type 2 machinery
    effective_from     DATE        NOT NULL,
    effective_to       DATE,
    is_current         BOOLEAN     NOT NULL DEFAULT TRUE
);

-- =====
-- Priya moves Hyderabad → Bengaluru → Chennai
-- Each move: close old row, insert new row, AND overwrite
-- current_city on ALL rows of this customer_id.
-- =====

-- Step 1: close old current row
UPDATE dim_customer
SET     effective_to = DATE '2026-07-14', is_current = FALSE
WHERE  customer_id = 'C-123' AND is_current = TRUE;

-- Step 2: insert new current row (Type 2)
INSERT INTO dim_customer
(customer_key, customer_id, full_name,
 city, current_city, previous_city,
 effective_from, effective_to, is_current)
SELECT nextval('dim_customer_seq'),
```

```

'C-123', 'Priya K',
'Chennai',          -- new point-in-time value
'Chennai',          -- current_city (will be set correctly
below)
'Bengaluru',        -- immediately prior
DATE '2026-07-15', NULL, TRUE;

-- Step 3: Type 1 overwrite – update current_city on ALL
-- historical rows of this customer_id to the latest value.
UPDATE dim_customer
SET    current_city = 'Chennai'
WHERE  customer_id = 'C-123';

-- =====
-- Reports can pick 'as-was' or 'as-is' from the SAME row
-- =====

SELECT s.order_id,
       c.city          AS as_was_city,    -- Type 2 (historical)
       c.current_city AS as_is_city,     -- Type 1 (latest)
       s.net_amount
FROM   fact_sales s
JOIN   dim_customer c ON c.customer_key = s.customer_key
WHERE  c.customer_id = 'C-123'
ORDER BY s.order_date;
```

Student

What is **SCD Type 6**?

ChatGPT • AI Assistant

Type 6 = Type 1 + Type 2 + Type 3 all in one dimension. (The number 6 was chosen because $1 + 2 + 3 = 6$.) It gives you **both** point-in-time historical accuracy AND easy "latest value" reporting from the same dimension.

CONCEPT CARD

The three pieces:

1. **Type 2 versioning** - each change creates a new row with new surrogate key (as-was accuracy).
2. **Type 1 overwrite of "current" column** - every row also carries a column like **current_city** that reflects the LATEST value across all versions.

3. Type 3 previous column - optional "most recent prior value" column on each row.

Student

How does "current on every row" work?

ChatGPT • AI Assistant

EXAMPLE CARD

Priya moves from Hyderabad → Bengaluru → Chennai. Under Type 6:

1. Row 1: `city='Hyderabad'` , `current_city='Chennai'` (updated on every change), `effectivefrom='2026-01-01'`, `iscurrent=false`.
2. Row 2: `city='Bengaluru'` , `current_city='Chennai'` , `is_current=false`.
3. Row 3: `city='Chennai'` , `current_city='Chennai'` , `is_current=true`.

Now:

- Historical fact rows joined to row 1 see `city='Hyderabad'` (as-was) AND `current_city='Chennai'` (as-is).
- One query, both perspectives, no self-join needed.

🔗 ANALOGY

Type 6 is like a **photo album** (Type 2) where every photo has a **sticky note** with today's info (Type 1 on current_) and a **caption** noting the previous state (Type 3). Best of all worlds - at the cost of ETL complexity.

⚠️ WARNING

Type 6 is more complex to maintain. Every time an attribute changes, ETL must update EVERY row for that natural key to refresh the `current_` columns. Cost per change is higher than pure Type 2.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Building Type 6 for every attribute in a dimension. ETL becomes a maintenance nightmare.

Correct Way: Type 6 selectively - only on attributes where BOTH as-was AND as-is analysis is genuinely needed (typically 1-3 columns per dim).

EXAM TIP CARD

Interview classic: "Why is it called SCD Type 6?" Winning answer: "Because it combines Types 1, 2, and 3 - and $1+2+3=6$. The dimension carries the historical

value (Type 2), a same-row 'current' snapshot (Type 1 overwrite), and optionally a 'previous' column (Type 3)."

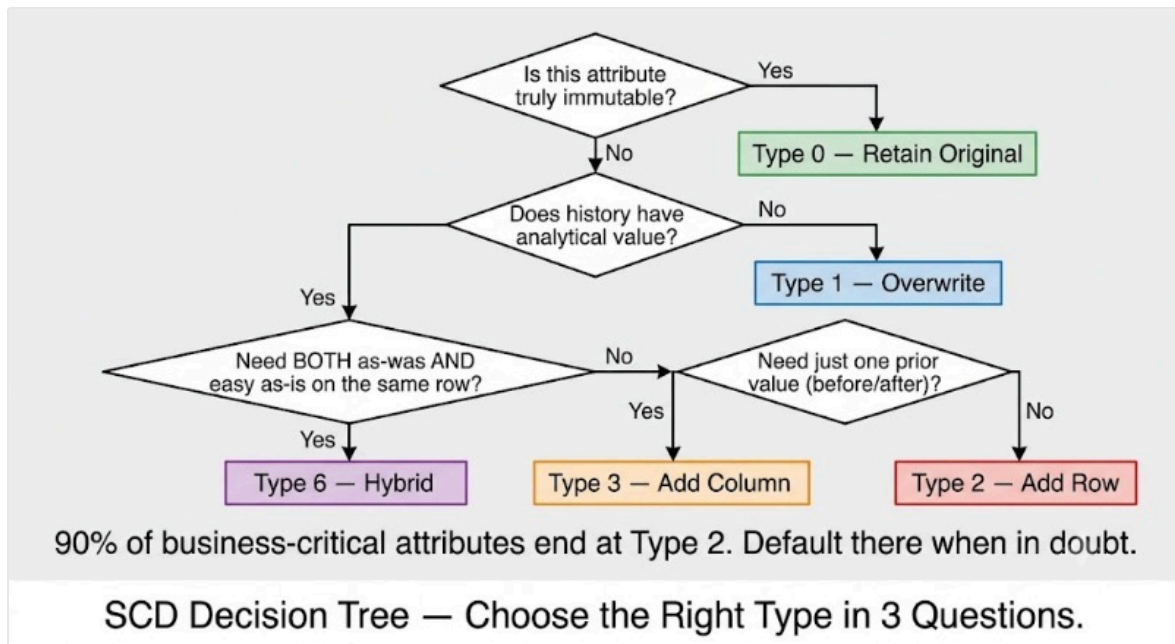
🔥 EXERCISE

Take `dimcustomer` with a `'city'` attribute. Add a `'currentcity'` column that's Type-1-overwritten on every change. Simulate 3 moves. Write a query returning both the as-was and the as-is city for each of Priya's orders.

[AI Prompt] Paste this into any AI chatbot: "Show me a Type 6 `dim_customer` with columns for as-was city (Type 2), current city (Type 1 overwritten across all rows), and previous city (Type 3). Include the ETL update to change city, and one report using each perspective."

CHOOSING THE RIGHT SCD TYPE

ChatGPT • AI Assistant



SCD Decision Tree — Choose the Right Type in 3 Questions

Student

With so many types, how do I **choose** for each attribute?

ChatGPT • AI Assistant

Use a **decision tree of three questions**. Ask them in order for every column in every dimension. Ten minutes of thinking here saves months of debugging bad reports later.

CONCEPT CARD

The three-question decision tree:

1. **Is this attribute truly immutable?** (Birth date, VIN, original signup date.) → **Type 0**.
2. **Does history have analytical value?** *If no* (typos, cosmetic changes, corrections) → **Type 1**. *If yes* → continue.
3. **Do I need BOTH 'as-was' and easy 'as-is' from the same row?** *If yes* → **Type 6**. *If just before/after* → **Type 3**. *Otherwise* → **Type 2**.

Student

Any shortcuts?

ChatGPT • AI Assistant

EXAMPLE CARD

Rules of thumb:

1. **Default to Type 2** for critical attributes (city, segment, category) - history is cheap; wrong reports are expensive.
2. **Type 1** for low-value fields (email typos, cosmetic name changes, secondary phone).
3. **Type 0** only for provably immutable fields (birth date, first-signup date, VIN).
4. **Type 3** only for one-time transitions (rare).
5. **Type 6** only when both perspectives are demanded by real reports.

🔗 ANALOGY

Think of it like **backing up files**. Type 0 = write-once files (never overwritten). Type 1 = no backup (overwrite in place). Type 2 = full version history. Type 3 = save one prior copy. Type 6 = full history + a shortcut to the latest.

⚠️ WARNING

Don't over-engineer. Every dimension having every column as Type 6 makes ETL brittle. Pick the simplest type that meets the business need.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Blanket Type 2 across every column. ETL becomes complex and dimensions balloon.

Correct Way: Per-attribute choice. Most dims are a mix: a few Type 0 columns, many Type 1, a few Type 2 (the analytically important ones), rare Type 3 or 6.

EXAM TIP CARD

Interview classic: *"Would you make every attribute Type 2?"* Winning answer: *"No - Type 2 has a maintenance cost. I choose per attribute using: is it immutable (Type 0), does history matter (yes = Type 2, no = Type 1), do we need both perspectives (Type 6).*

Most dims are a mix."*

🔥 EXERCISE

For your *dimcustomer*, decide the SCD type for each of these attributes: *customerid*, *name*, *email*, *city*, *segment*, *birthdate*, *signupdate*, *lastlogindate*, *loyalty_tier*. Justify each choice.

[AI Prompt] Paste this into any AI chatbot: *"For dim_employee with columns emp_id, name, email, birth_date, hire_date, department, manager, job_title, and salary_band, recommend the SCD type for each. Justify each choice."*

IMPLEMENTING SCD TYPE 2 IN ETL

ChatGPT • AI Assistant

```
sql - Full Type 2 ETL - change detection + close + insert
```

```
-- =====
-- SCD Type 2 ETL for dim_customer
-- Assumes staging_customer has fresh data from the source.
-- Type 2 columns: city, segment
-- Type 1 columns: email, full_name
-- =====

-- Step 1: Detect Type 2 changes
-- (compare tracked-column hash between staging and current dim)
WITH changed AS (
    SELECT s.customer_id, s.city, s.segment,
```

```

        s.email, s.full_name
FROM    staging_customer s
JOIN    dim_customer      d
      ON  d.customer_id = s.customer_id
      AND d.is_current   = TRUE
WHERE   MD5(s.city || '|' || s.segment)
        <> MD5(d.city || '|' || d.segment)
)

-- Step 2: Close the outdated current rows
UPDATE dim_customer
SET    is_current   = FALSE,
       effective_to = CURRENT_DATE - 1,
       updated_ts   = CURRENT_TIMESTAMP
WHERE  customer_id IN (SELECT customer_id FROM changed)
      AND is_current = TRUE;

-- Step 3: Insert the new versions
INSERT INTO dim_customer (
    customer_key, customer_id, full_name, email,
    city, segment,
    effective_from, effective_to, is_current, version_number
)
SELECT nextval('dim_customer_seq'),
       c.customer_id, c.full_name, c.email,
       c.city, c.segment,
       CURRENT_DATE, NULL, TRUE,
       COALESCE(
           (SELECT MAX(version_number) FROM dim_customer
            WHERE customer_id = c.customer_id), 0
       ) + 1
FROM    changed c;

-- Step 4: Insert brand-new customers (never seen before)
INSERT INTO dim_customer (
    customer_key, customer_id, full_name, email,
    city, segment,
    effective_from, effective_to, is_current, version_number
)
SELECT nextval('dim_customer_seq'),
       s.customer_id, s.full_name, s.email,
       s.city, s.segment,
       CURRENT_DATE, NULL, TRUE, 1
FROM    staging_customer s
LEFT JOIN dim_customer d
      ON  d.customer_id = s.customer_id

```

```

WHERE d.customer_id IS NULL;

-- Step 5: Apply Type 1 overwrites on unchanged-Type-2 rows
-- (email or name corrections that shouldn't create a new row)
UPDATE dim_customer d
SET    email      = s.email,
       full_name   = s.full_name,
       updated_ts  = CURRENT_TIMESTAMP
FROM    staging_customer s
WHERE   d.customer_id = s.customer_id
       AND d.is_current = TRUE
       AND MD5(d.city || '|' || d.segment)
          = MD5(s.city || '|' || s.segment)           -- no Type 2 change
       AND (d.email <> s.email OR d.full_name <> s.full_name);

```

Student

How do I actually **implement Type 2 in an ETL pipeline?**

ChatGPT • AI Assistant

The universal pattern is called the **Type 2 MERGE**. For each row in the source, you check if the natural key already exists in the dim.

If yes but with a **changed tracked attribute**, close the old row and insert a new one. If yes with no change, do nothing.

If not there, insert as a fresh version.

CONCEPT CARD

The Type 2 MERGE algorithm in 4 branches:

1. **New natural key** - INSERT with *versionnumber=1, iscurrent=true*.
2. **Existing key + no change** - do nothing (idempotent).
3. **Existing key + tracked attribute changed** - UPDATE close old (*iscurrent=false, effectiveto=change_date - 1*), then INSERT new version.
4. **Existing key + only Type 1 attribute changed** - UPDATE the existing row's Type 1 columns in place.

Student

Do I use a single SQL MERGE statement?

ChatGPT • AI Assistant

EXAMPLE CARD

Yes - modern warehouses support **MERGE** with **WHEN MATCHED** and **WHEN NOT MATCHED**. But Type 2's "close + insert" is really **two writes**, so most ETL frameworks split it into an UPDATE step (close old rows) followed by an INSERT step (add new versions).

Tools like **dbt snapshots** handle all of this declaratively.

🔗 ANALOGY

Type 2 ETL is like **updating a book in a library**. You don't rip out and rewrite pages - you file the old edition in the archive (close it), then add the new edition on the current shelf (insert new row). Both editions remain findable.

⚠️ WARNING

Watch out for these ETL bugs:

1. **Change-detection false positives** - trailing whitespace or case differences trigger unnecessary new rows.
2. **Late-arriving data** - an update dated in the past can invalidate later versions. Handle carefully.
3. **Duplicate natural keys** in staging - accidentally create two "current" rows.
4. **Type 1 columns updated but Type 2 columns unchanged** - must be handled without a new row.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Using a plain UPSERT that overwrites the existing row in place - you just accidentally applied Type 1 to a Type 2 column.

Correct Way: Separate the logic. Close the old row via UPDATE, then INSERT the new row with a new surrogate key.

EXAM TIP CARD

Interview classic: *"How do you implement SCD Type 2 in ETL?"* Winning answer:

*"Detect changes on tracked columns. Close the current row by setting *iscurrent=false and stamping effective to*.

Insert a new row with a new surrogate key and *effective_from*. Idempotency comes from a hash of tracked columns."*

EXERCISE

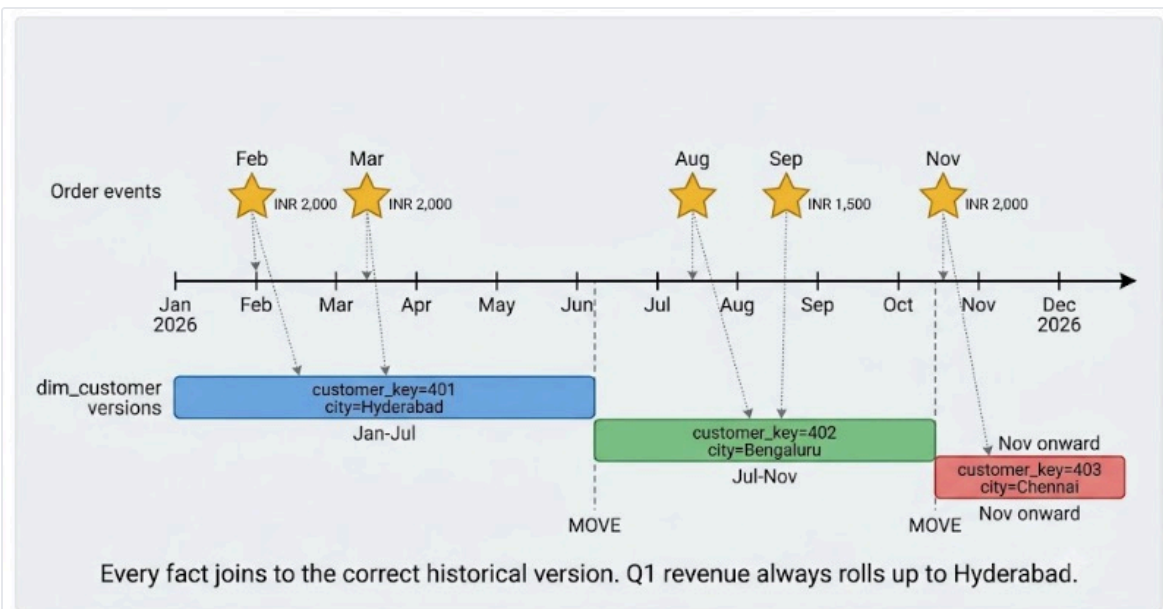
Write a MERGE-based ETL for `dim_customer` treating city as Type 2 and email as Type 1. Test it with three staging batches: a new customer, an unchanged customer, and a customer who moved.

[AI Prompt] Paste this into any AI chatbot: `*"Write a Snowflake / PostgreSQL SQL script that implements SCD Type 2 for dim_customer. Include: change detection via hash, UPDATE that closes the old row, INSERT for the new row.`

`Assume 'city' and 'segment' are Type 2, 'email' is Type 1."*`

HANDS-ON — TRACK CUSTOMER ADDRESS CHANGES

ChatGPT • AI Assistant



Priya's Timeline — Facts, Dim Versions, and the Star Join.

Priya's Timeline — Facts, Dim Versions, and the Star Join

sql — Complete SCD Type 2 hands-on — Priya's moves across 2026

```
-- =====
-- Hands-on: SCD Type 2 for customer address changes
-- Story: Priya buys in Hyderabad, moves to Bengaluru,
--        then to Chennai. History is preserved.
-- =====
-- dim_customer with SCD Type 2 machinery
```

```
CREATE TABLE dim_customer (  
    customer_key    BIGINT        PRIMARY KEY,  
    customer_id     VARCHAR(50)   NOT NULL,  
    full_name       VARCHAR(200),  
    city            VARCHAR(100),  
    effective_from  DATE          NOT NULL,  
    effective_to    DATE,  
    is_current      BOOLEAN       NOT NULL DEFAULT TRUE  
);  
  
-- Minimal fact table  
CREATE TABLE fact_sales (  
    sales_key       BIGINT        PRIMARY KEY,  
    customer_key    BIGINT        REFERENCES dim_customer(customer_key),  
    order_date      DATE          NOT NULL,  
    net_amount      DECIMAL(12, 2) NOT NULL  
);  
  
-- =====  
-- STEP 1: Priya signs up in Hyderabad on Jan 1, 2026  
-- =====  
INSERT INTO dim_customer VALUES  
    (401, 'C-123', 'Priya K', 'Hyderabad',  
     DATE '2026-01-01', NULL, TRUE);  
  
-- Two orders while in Hyderabad  
INSERT INTO fact_sales VALUES  
    (1, 401, DATE '2026-02-14', 2500.00),  
    (2, 401, DATE '2026-03-22', 1800.00);  
  
-- =====  
-- STEP 2: Priya moves to Bengaluru on Jul 15, 2026  
-- =====  
UPDATE dim_customer  
SET     effective_to = DATE '2026-07-14',  
        is_current   = FALSE  
WHERE   customer_id = 'C-123' AND is_current = TRUE;  
  
INSERT INTO dim_customer VALUES  
    (402, 'C-123', 'Priya K', 'Bengaluru',  
     DATE '2026-07-15', NULL, TRUE);  
  
-- Two orders while in Bengaluru  
INSERT INTO fact_sales VALUES  
    (3, 402, DATE '2026-08-05', 3200.00),  
    (4, 402, DATE '2026-09-18', 4100.00);
```

```

-- =====
-- STEP 3: Priya moves to Chennai on Nov 1, 2026
-- =====

UPDATE dim_customer
SET     effective_to = DATE '2026-10-31',
        is_current   = FALSE
WHERE  customer_id = 'C-123' AND is_current = TRUE;

INSERT INTO dim_customer VALUES
(403, 'C-123', 'Priya K', 'Chennai',
 DATE '2026-11-01', NULL, TRUE);

-- One order in Chennai
INSERT INTO fact_sales VALUES
(5, 403, DATE '2026-11-20', 2900.00);

-- =====
-- REPORT 1: 'As-is' – where is Priya NOW?
-- =====

SELECT c.customer_id, c.city AS current_city
FROM   dim_customer c
WHERE  c.customer_id = 'C-123'
      AND c.is_current = TRUE;

-- =====
-- REPORT 2: 'As-was' – Q1 revenue by city (should be Hyderabad!)
-- =====

SELECT c.city,
       SUM(s.net_amount) AS q1_revenue
FROM   fact_sales      s
JOIN   dim_customer    c ON c.customer_key = s.customer_key
WHERE  s.order_date BETWEEN DATE '2026-01-01' AND DATE '2026-03-31'
GROUP BY c.city;
-- Result: Hyderabad, INR 4,300.  Historically correct ✓

-- =====
-- REPORT 3: Revenue by city across all quarters
-- =====

SELECT c.city,
       DATE_TRUNC('quarter', s.order_date) AS quarter,
       SUM(s.net_amount) AS revenue
FROM   fact_sales      s
JOIN   dim_customer    c ON c.customer_key = s.customer_key
GROUP BY c.city, quarter
ORDER BY quarter, c.city;

```

```
-- =====  
-- Key insight: fact rows are linked to the SURROGATE KEY of  
-- the dim version that was current at the time of the sale.  
-- History stays honest – no matter how many times Priya moves.  
-- =====
```

Student

Can you walk me through a **complete hands-on** for a customer moving between cities under different SCD types?

ChatGPT • AI Assistant

Perfect capstone for this chapter. We'll track **Priya (customer_id C-123)** as she moves from **Hyderabad** → **Bengaluru** → **Chennai** across three quarters. We'll show what happens under each SCD type and what queries look like against each.

CONCEPT CARD

The scenario:

1. **Jan 2026** - Priya signs up in Hyderabad. Buys twice (Feb, Mar).
2. **Jul 15, 2026** - Priya moves to Bengaluru. Buys twice (Aug, Sep).
3. **Nov 1, 2026** - Priya moves to Chennai. Buys once (Nov).

Question: "**How much revenue came from Hyderabad customers in Q1?**"

- Under Type 1 (overwrite): Priya's Q1 orders now roll up to her CURRENT city (Chennai) - Hyderabad revenue is understated.
- Under Type 2 (versioning): Priya's Q1 orders roll up correctly to Hyderabad.

Student

Show me the DDL and inserts.

ChatGPT • AI Assistant

EXAMPLE CARD

The code block below is a complete, runnable script:

- 1. Creates `dim_customer` with SCD Type 2 columns.
- 2. Inserts Priya's initial row (Hyderabad).
- 3. Simulates two moves with the close-old + insert-new pattern.
- 4. Inserts 5 `fact_sales` rows across the timeline.
- 5. Runs three reports: as-is, as-was, and revenue-by-city-quarter - showing the correct historical accuracy.

🔗 ANALOGY

This hands-on is your **certificate of comprehension** for SCD Type 2. If you can read and modify this script, you can implement historical tracking in any warehouse.

⚠️ WARNING

The example uses simple manual INSERTs to keep it clear. In production, all this is driven by ETL tools (dbt snapshots, Airflow, Fivetran) that handle change detection and MERGE for you.

🔥 EXERCISE

Copy the DDL and inserts into any SQL environment. Run each report. Then try modifying: what if Priya moves back to Hyderabad in Dec? What if her segment changes from 'new' to 'vip' on Aug 1? Extend the script yourself.

[AI Prompt] Paste this into any AI chatbot: *"Extend this SCD Type 2 hands-on: add a customer_segment column that also uses Type 2. Simulate a segment change from 'new' to 'loyal' on 2026-06-01. Show the resulting dimension rows and re-run the as-was Q1 revenue query."*

CHAPTER REVIEW & SUMMARY

Student

Can you explain what this chapter is about?

ChatGPT • AI Assistant

CHAPTER SUMMARY

This chapter solved the biggest operational problem in dimensional modeling: how to handle history when dimension attributes change. You now have the full SCD toolkit and know when to reach for each.

- **WHAT IS A SLOWLY CHANGING DIMENSION:** A dimension whose attribute values change over time - customer moves, product recategorized, employee transferred. The 'slowly' contrasts dimension change frequency with fact table growth.
- **SCD TYPE 0 - RETAIN ORIGINAL:** Never change. Use for truly immutable attributes like birth date, VIN, original signup date. Enforced at the ETL level.
- **SCD TYPE 1 - OVERWRITE:** Simple UPDATE. No history preserved. Use for corrections and cosmetic changes where history has no analytical value.
- **SCD TYPE 2 - ADD ROW WITH HISTORY:** The workhorse. Every change creates a new row with a new surrogate key; the old row is closed via *iscurrent=false and effectiveto*. Historical facts stay joined to their correct version. This is why we use surrogate keys.
- **SCD TYPE 3 - ADD COLUMN FOR PREVIOUS VALUE:** Store one prior value in a separate column. Useful for one-off transitions like sales-territory realignments - before/after only, not full history.
- **SCD TYPE 6 - HYBRID:** Type 1 + Type 2 + Type 3 combined (1+2+3=6). Every row has both the point-in-time value AND the always-latest 'current_' column. Best of both worlds at extra ETL cost.
- **CHOOSING THE RIGHT SCD TYPE:** Three-question decision tree - immutable (Type 0), history matters (Type 1 vs deeper), need both perspectives (Type 6). Default to Type 2 when in doubt; most dims are a mix.
- **IMPLEMENTING SCD TYPE 2 IN ETL:** The Type 2 MERGE - detect tracked-column changes via hash, close old rows via UPDATE, insert new versions via INSERT, apply Type 1 overwrites separately for unchanged Type-2 rows.
- **HANDS-ON - TRACK CUSTOMER ADDRESS CHANGES:** A complete runnable script showing Priya's moves across Hyderabad → Bengaluru → Chennai with three preserving-history dim rows and correct 'as-was' Q1 revenue.

Student

Can you break down the next concept?

ChatGPT • AI Assistant

You now understand why every dimension you've seen in this refcard has *effectivefrom* / *effectiveto* / *is_current* columns - and how to actually use them in production ETL. In the next chapter we step back and look at the big picture: **Star and**

Snowflake Schemas - when to denormalize a dim fully vs when to normalize part of it out, and how the choice affects query performance and maintainability.